robot-control Plugin Tools

robot-control Plugin Tools
1. Course Content
   Course Overview
   Learning Objectives
2. Preparation
3. OpenClaw Plugins
4. Introduction to the robot-control Plugin
5. Plugin Tool Management
6. Source Code Analysis
   6.1 Plugin Entry Point and Tool Registration
   6.2 MCP Client Communication

1. Course Content

Course Overview

- The robot-control plugin is the core robot control plugin of OpenClaw. As a bridge between OpenClaw AI and robot hardware, it converts users' natural language instructions into specific robot operation commands through the MCP protocol.

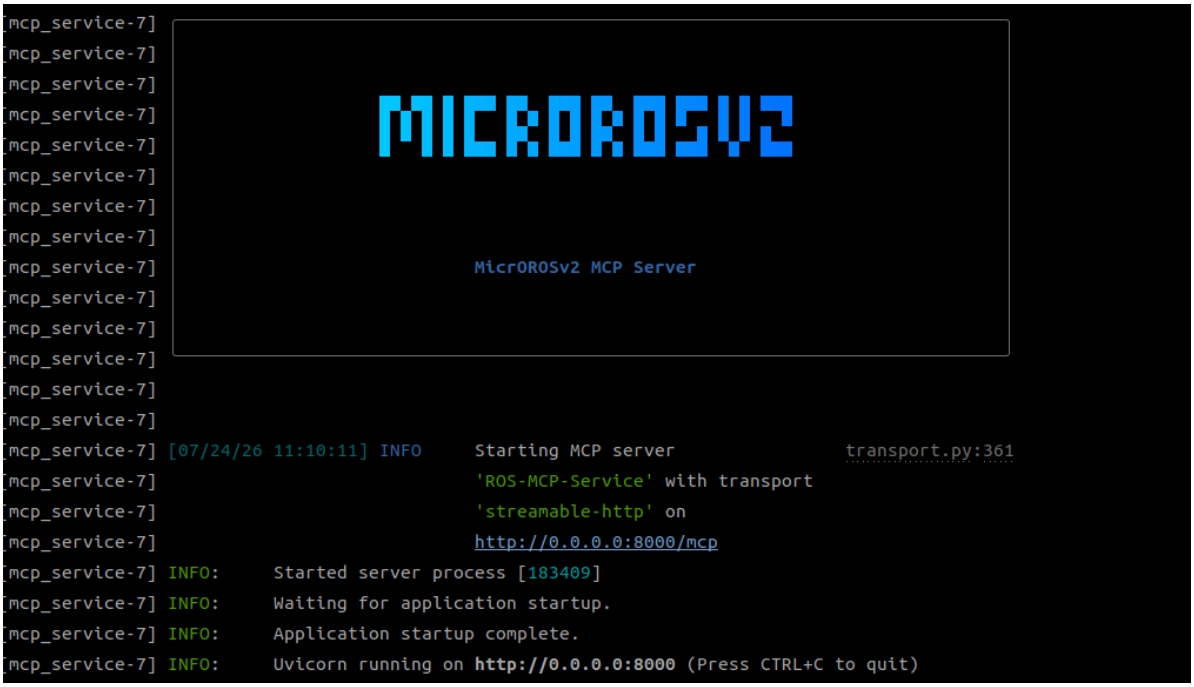
Learning Objectives

1. Understand the role and functional positioning of the robot-control plugin in the OpenClaw framework
2. Master the installation, viewing, and basic management operations of the plugin
3. Be able to manage and call robot control tools through the OpenClaw WebChat interface

> [!IMPORTANT]
>
> The demo in this lesson uses the pan-tilt camera version vehicle as an example; the MCP and CLI tool types may differ slightly across different vehicle versions

2. Preparation

```
ros2 launch microros_v2_bringup openclaw_service.launch.py
```



3. OpenClaw Plugins

- Plugin function introduction: The OpenClaw plugin system is a functional extension mechanism of the OpenClaw platform. By installing different plugins, you can add various capabilities to OpenClaw. Currently, the core plugins related to robot control include:
- **Robot-Control Plugin**: The core robot control plugin, providing all robot operation functions such as chassis movement, robotic arm control, visual recognition, and navigation and positioning

```
openclaw plugins list
```

- In the plugin list, Robot-Control is the plugin used to control the robot with OpenClaw

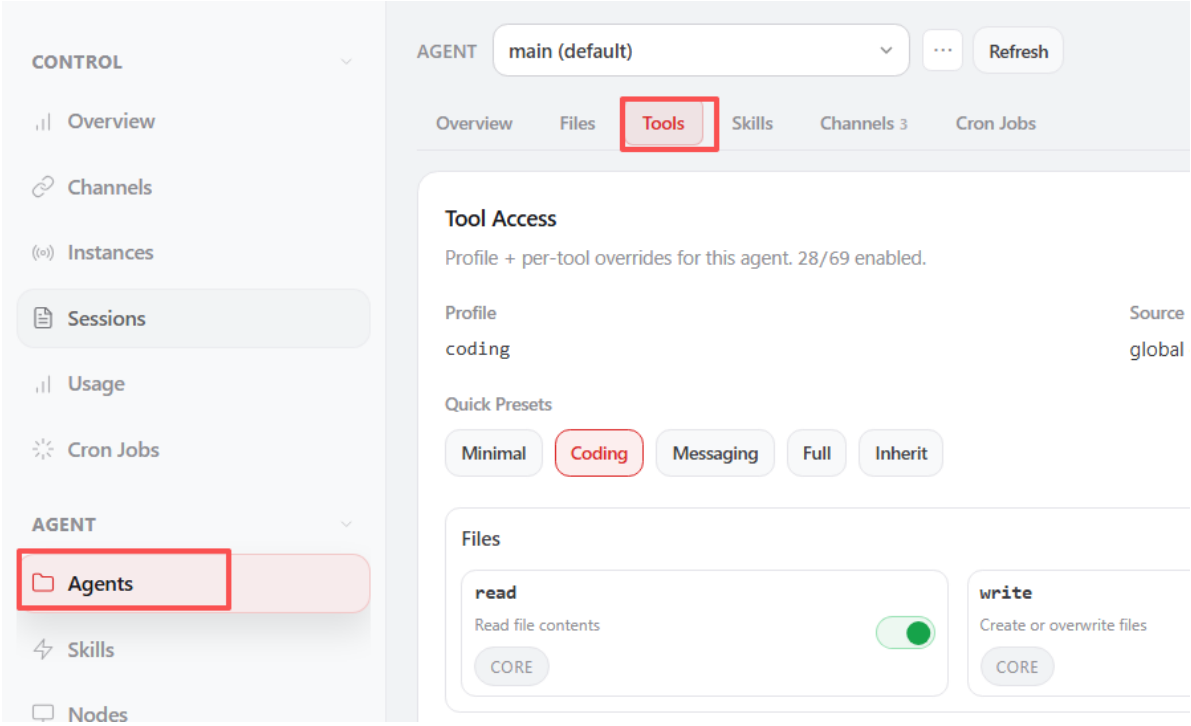
4. Introduction to the robot-control Plugin

The robot-control plugin is the core robot control plugin. Its working principle is as follows:

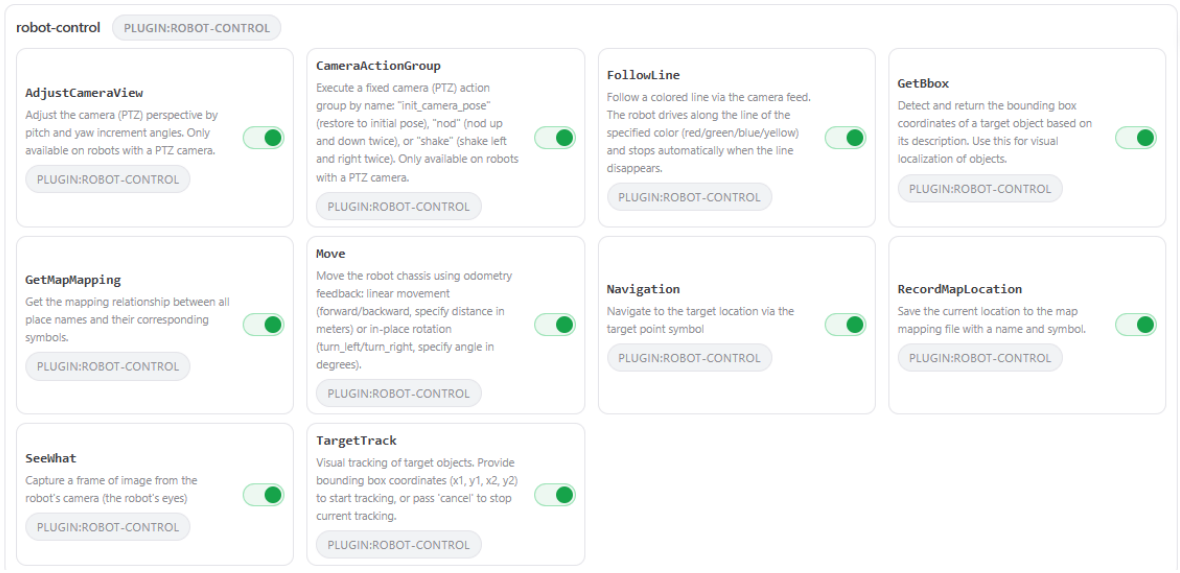
- **Communication mechanism**: After the LLM in OpenClaw parses the user's natural language instructions, it calls the MCP protocol interface through the robot-control plugin, passes the instructions to the MCP server, and the MCP server then controls the robot hardware to perform actions through the ROS2 communication stack

5. Plugin Tool Management

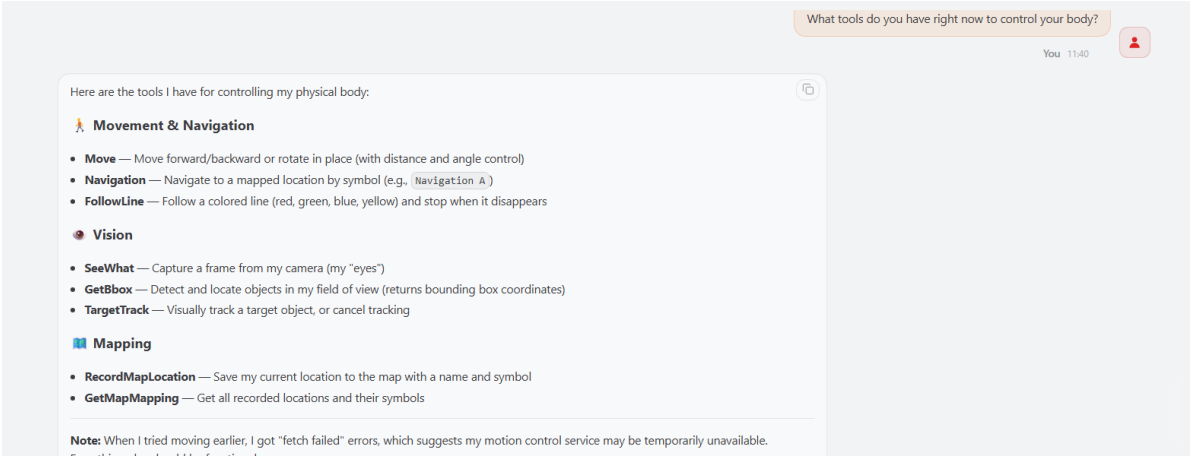
- Log in to the OpenClaw WebChat interface, then select Agent — Tools in sequence



- Scroll down to find the robot-control plugin tool list to view and manage all plugin tools
    - All plugin functions of robot-control are enabled by default at the factory (corresponding to the MCP and CLI interfaces). If you need to manually disable a tool, click to turn off the corresponding tool, click Save, and restart the OpenClaw gateway



- You can directly ask OpenClaw to view the currently available robot control tools



6. Source Code Analysis

6.1 Plugin Entry Point and Tool Registration

Source Code Directory:

```
$HOME/.openclaw/extensions/robot-control/index.ts
```

The plugin source code is located in `index.ts` in the plugin directory. It uses the `registerRobotControlTools` function as the entry point and uses the OpenClaw Plugin SDK to register all robot control tools. Take the `Move` (chassis movement) tool as an example:

```
export default function registerRobotControlTools(api: OpenClawPluginApi) {
  // Load the SeeWhat channel config at startup
  void loadSeeWhatConfig(api.config as Record<string, unknown> | undefined);

  api.registerTool(
    {
      name: "Move",
      label: "Move",
      description: "Control the robot's chassis to move a specified distance.",
      parameters: MoveSchema,
      async execute(_toolCallId, params) {
        try {
          const args = buildArgs({ "distance": params.distance, "direction":
params.direction });
          const result = await callRobotControl("Move", args);
          return json(result);
        } catch (err) {
          return json({ error: err instanceof Error ? err.message : String(err)
});
        }
      },
    },
    { name: "Move" },
  );
  // ... registration of other tools
}
```

Code explanation:

1. `api.registerTool()`: The tool registration method provided by the SDK; it takes a tool definition (name, description, parameters Schema, execute function) and metadata
2. **Parameters Schema**: Uses `Type.Object()` from `@sinclair/typebox` to define the parameter structure and constraints
3. **Execute function**: `buildArgs` converts the parameters into CLI format, and `callRobotControl` calls the robot control interface

6.2 MCP Client Communication

The plugin internally encapsulates the `McpClient` class, which communicates with the MCP server through the HTTP protocol using JSON-RPC:

```
const MCP_ENDPOINT = "http://localhost:8000/mcp";

class McpClient {
  private sessionId: string | null = null;
  private initialized = false;

  async ensureInitialized(): Promise<void> {
    if (this.initialized) return;
    await this.sendRequest("initialize", {
      protocolVersion: "2025-03-26",
      capabilities: {},
      clientInfo: { name: "openclaw-plugin", version: "1.0.0" },
    });
    await this.sendNotification("notifications/initialized");
    this.initialized = true;
  }

  async callTool(name: string, args: Record<string, unknown>) {
    await this.ensureInitialized();
    return await this.sendRequest("tools/call", { name, arguments: args });
  }
}
```

Code explanation:

1. `McpClient`: Maintains the MCP session state. It is automatically initialized on the first call and sends tool call requests through the JSON-RPC protocol
2. `callRobotControl` **helper function**: Encapsulates the complete tool call flow and converts CLI-style parameters into the MCP parameter format